Problem Situation - Evidence 2

Aldo Karim Garcia Zapata
Departamento de Computación, Tecnológico de Monterrey
Monterrey, México
A00843726@tec.mx

Botnet attacks have multiplied to the point of causing major disruptions across all sectors. A typical trace of these attacks is a server log with thousands of failed access attempts coming from many IP addresses. This report describes a C++ solution that reads a log of 16,806 records, stores them in a doubly linked list, and sorts them by IP address and date/time with an iterative Quicksort. The chosen data structures and algorithms are justified, their best- and worst-case complexity is analyzed, the suitability of the linked list is discussed, and improvements are proposed.

I. WHAT IS THE PROBLEM, AND WHY IS IT RELEVANT?

A. The Problem

According to the FBI, cyberattacks carried out with botnets have multiplied to the point of causing major disruptions across all sectors. A botnet is a network of infected devices that an attacker controls remotely to launch coordinated actions, such as brute-force attacks against remote access services (SSH).

Each attempt leaves a line in the server log with this format:

Oct 9 10:32:24 423.2.230.77:6166 Failed password for illegal user guest

The fields are: month, day, time, IP:port, and message. The file contains 16,806 records in random order, so it is impossible to tell at a glance:

- Which IP tried to access the most times (possible bots).
- The chronological sequence of attempts from a single IP.
- Efficient lookup of the records of one IP or a range of IPs.

B. Relevance

Detection: an IP with hundreds of consecutive failures within seconds is a classic indicator of a bot. Grouping by IP and sorting by time makes it evident.

Incident response: forensic analysis needs an ordered timeline to know when the attack began and which accounts were targeted .

Scale: real logs grow to millions of lines, so the choice of data structures and algorithms directly affects analysis time.

Current context: law enforcement operations against botnets show that the problem is persistent and that many always remain active.

II. HOW WAS THE PROBLEM SOLVED?

The solution is divided into four modules

Module	**File**	**Responsibility**
Record	Entry.h/.cpp	Models a log line and computes its numeric timestamp.
Keeper	Keeper.h/.cpp	Doubly linked list that reads and stores the records.
Sorter	Sorter.h/.cpp	Sorts by IP and then by date/time (iterative Quicksort).
Searcher	Searcher.h	Reserved for search.

Reading (Keeper::read): bitacora.txt is opened and the month, day, time, and IP are read with >>; the rest of the line is taken with getline as the message. Each record is inserted at the end of the list with pushBack.

Entry construction: the constructor converts month + day + time into a number timestamp = month·10^8 + day·10^6 + hour·10^4 + minute·10^2 + second, which allows comparing dates with a single integer comparison.

Sorting (Sorter::sort): each IP is converted to a number (ipToNumber), the records are copied into a vector<Item>, sorted with iterative Quicksort, and the result is written back onto the list nodes, preserving the structure.

Output: main.cpp currently prints the number of records read; the sorted result stays in the list, ready for searches or reports.

III. DATA STRUCTURES AND ALGORITHMS

A. Doubly Linked List (Keeper)

Implementation: each Node stores an Entry data and next and prev pointers. The Keeper class maintains head, tail, and count.

```
void Keeper::pushBack(Entry data) {
   Node *node = new Node(data);
   if (head == nullptr) { head = tail = node; }
   else { tail->next = node; node->prev = tail; tail = node; }
   count++;
}
```

Why it was chosen:

- Records arrive as a stream and their total number is unknown at the start; the list grows without reserving or copying memory.
- By storing tail, inserting at the end is truly O(1) (no list traversal).
- The prev pointer allows traversal in both directions, useful for range queries (from a start IP to an end IP) and for deleting nodes in O(1) given their pointer.
- It is the main change from Evidence 1, where two vectors were used; the struct keeps each record's data together.

B. Entry (record struct)

Stores the original fields and the precomputed timestamp.

C. Iterative Quicksort (Sorter)

Implementation:

- A stack<pair<int,int>> stores the pending (low, high) ranges to sort, instead of using recursion.
- The pivot is the middle element of the range
- Hoare partition moves two indices (i from the left, j from the right), swaps out-of-place elements, and ends when they cross. The two halves are then pushed onto the stack.
- Sorting is done on a vector<Item>, where Item contains ip (numeric), timestamp, and the Entry, so text is not reconverted on every comparison.

Why it was chosen:

- Quicksort: Very fast in practice and is the algorithm covered in class.
- Iterative: It is a requirement and avoids overflowing the call stack with deep recursion.
- Middle pivot: Avoids the worst case on already sorted or reversed data.
- Copy to vector: Quicksort needs O(1) random access, which the linked list does not offer.

IV. IS A LINKED LIST APPROPRIATE FOR THIS PROBLEM?

It is suitable for ingestion, but not for sorting or searching.

A. In Favor

- O(1) insertion at the end with a tail pointer, ideal for reading a file of unknown size.
- No reallocations: a vector may copy all its contents when growing; the list does not.
- O(1) deletion and insertion given a position (useful for filtering suspicious records).
- Double link: bidirectional traversal.

B. Against

- No random access: accessing element k costs O(k). Because of this, Quicksort cannot be applied directly and a copy to a vector was needed, adding O(n) memory and time.
- Binary search is impossible: searching for an IP in the sorted list is O(n); in a sorted array it would be O(log n).
- More memory: each node carries two pointers and one dynamic allocation per record.
- Poor cache locality: nodes are scattered in memory, making traversals slower than in a contiguous array.

C. Conclusion

The list works well as an ingestion container and allows sorting "in place", but for intensive analysis a dynamic array or a merge sort on the list would be more natural. With 16,806 records the practical difference is small; with millions it would be noticeable.

V. ADVANTAGES AND DISADVANTAGES

A. Doubly Linked List

Advantages	***Disadvantages***
O(1) insertion at the end	*O(n) access by position*
Grows dynamically without copying	*Higher memory use per node*
Traversal in both directions	*Worse cache locality*
O(1) deletion with node pointer	*Does not allow binary search*

B. Iterative Quicksort

Advantages	***Disadvantages***
Average O(n log n), very fast in practice	*Worst case O(n²)*
No recursion: no risk of overflowing the call stack	*Requires random access (forces copying the list)*
Handles repeated keys well	*The explicit stack can grow up to O(n) in the worst case*

VI. COMPUTATIONAL COMPLEXITY

Operation	**Best case**	**Average case**	**Worst case**	**Extra space**
pushBack	O(1)	O(1)	O(1)	O(1)
read (full read)	O(n)	O(n)	O(n)	O(1) in addition to the list
monthToNumber / Entry() / toString	O(1)	O(1)	O(1)	O(1)
ipToNumber	O(1)	O(1)	O(1) (bounded IP length)	O(1)
Destructor	O(n)	O(n)	O(n)	O(1)
Copy list → vector / rewrite	O(n)	O(n)	O(n)	O(n)
Iterative Quicksort	O(n log n)	O(n log n)	O(n²)	O(log n) to O(n) (stack)
Sorter::sort (total)	O(n log n)	O(n log n)	O(n²)	O(n)

A. What Determines Quicksort's Best and Worst Case?

The complexity depends on how balanced the partition is around the pivot:

- Best case O(n log n): the pivot splits the range into two nearly equal halves.
- Worst case O(n²): the pivot always lands at an extreme (the smallest or largest), producing partitions of size n-1 and 0.
- Average case O(n log n): with data in arbitrary order) the partitions are reasonably balanced.

B. Why the Stack Is O(log n) to O(n)

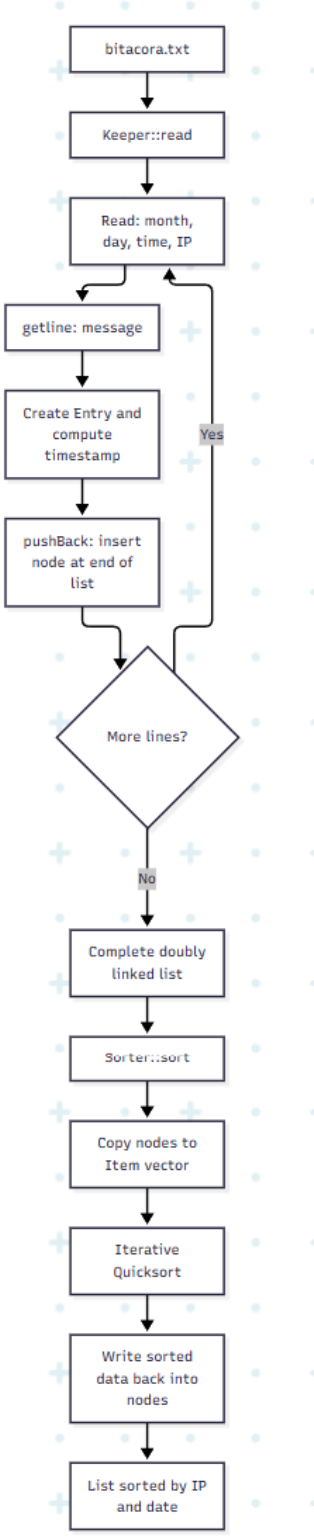
Both subranges are pushed without first choosing the smaller one. If the partitions are balanced, the stack holds O(log n) ranges; if they are highly unbalanced (worst case), it can reach O(n).

VII. POSSIBLE IMPROVEMENTS

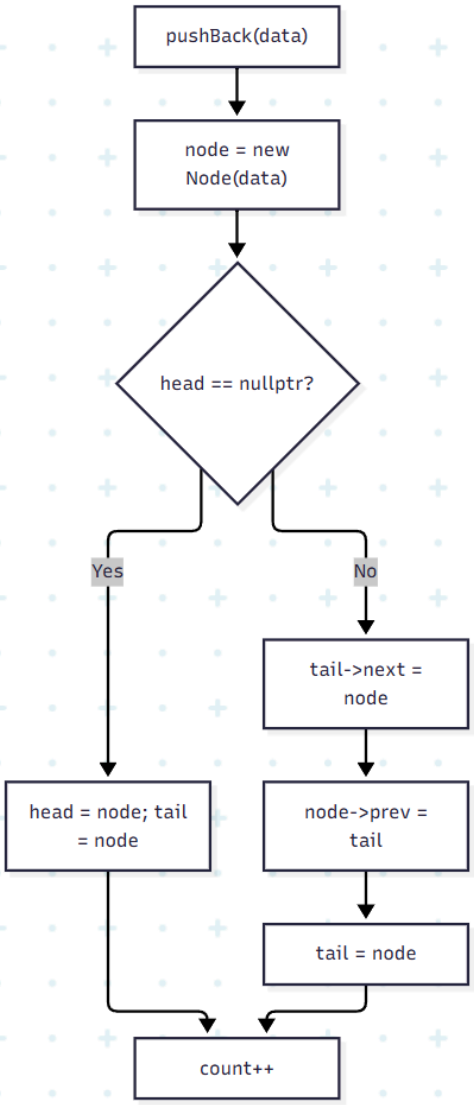
- Bot detection: once sorted, a single O(n) pass counts attempts per IP and flags those exceeding a threshold within a time window.
- Top offenders ranking: list the N IPs with the most failed attempts, with their first and last attempt times.
- Graphical visualization: a chart (bar or heat map) of attempts per IP or per hour, exported from the program.

VIII. OPERATION FLOW DIAGRAMS

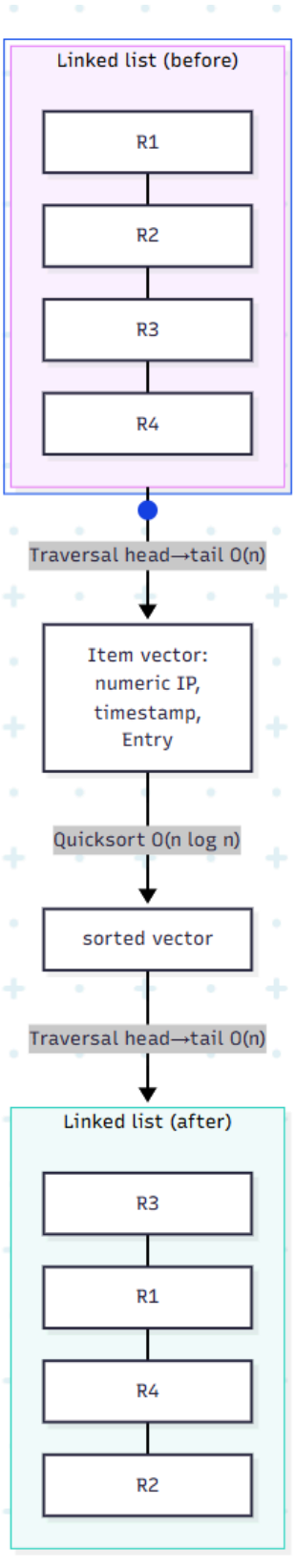
A. General Program Flow



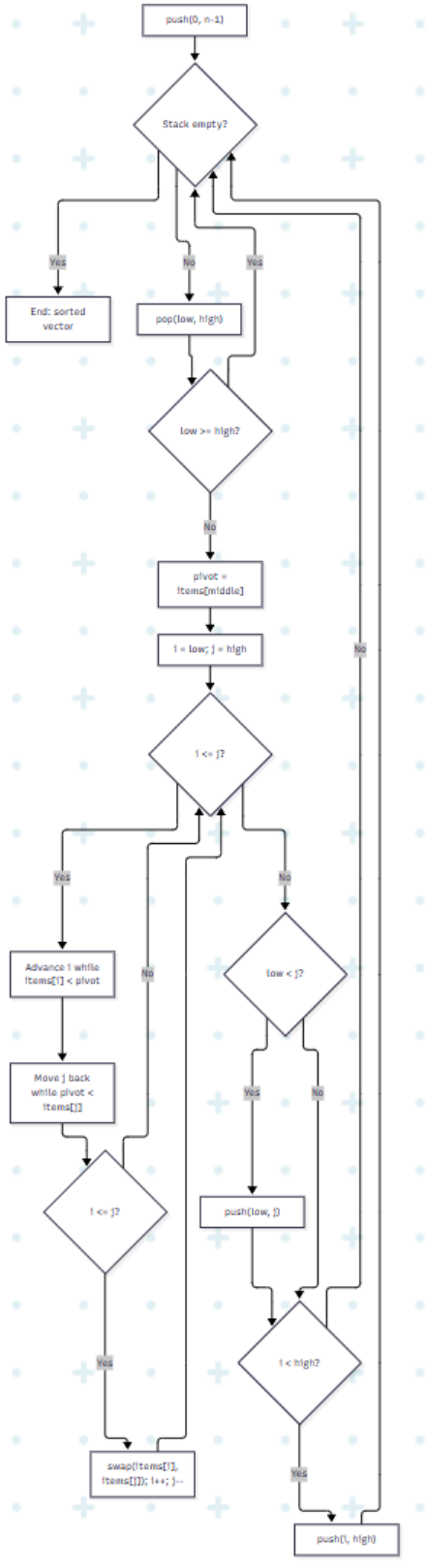
B. pushBack Operation on the Doubly Linked List



C. How the List Participates in Sorting



D. Iterative Quicksort with Range Stack



IX. CONCLUSIONS

The combination of a doubly linked list for ingestion and an iterative Quicksort makes it possible to sort a log of 16,806 records by IP and date, which helps identify botnet attack patterns. The list offers O(1) insertion and dynamic growth, but its lack of random access forces copying the data into a vector for sorting. Sorting is $O(n \log n)$ on average and $O(n^2)$ in the worst case. It would be interesting to add the improvements.

REFERENCES

[1] P. H. O'Neill, "The FBI shut down a huge botnet, but there are plenty more left," MIT Technology Review. [Online]. Available: https://www.technologyreview.com/s/604138/the-fbi-shut-down-a-huge-botnet-but-there-are-plenty-more-left/. [Accessed: Oct. 15, 2026].

AI usage: I used AI for better translating from Spanish to English and helping me to do the flowchart code.